

模拟卷 03-2025 风格题目与详解

目录

1	编译原理 2025 风格模拟卷 03（题目与详解）	2
2	A 卷题目	2
3	B 详解	4
4	C 复盘清单	9

1 编译原理 2025 风格模拟卷 03 (题目与详解)

1.1 卷面说明

本卷按 Slides/Slides/handout 与三份 Recap PPT 更新, 主线覆盖前端自动机与 LL、中端 IR/SSA、数据流、符号执行、后端目标代码、寄存器分配、指令调度以及过程间/指针分析。总分 100 分。

2 A 卷题目

2.1 一、词法分析 (15 分)

正则表达式:

$$(a|b)^* a a$$

1. 使用 Thompson 构造法构造等价 NFA。
2. 使用子集构造法构造 DFA, 写明 DFA 状态含义。
3. 最小化 DFA。

2.2 二、LL 语法分析 (15 分)

文法:

$$\begin{aligned} R &\rightarrow T U \\ T &\rightarrow T p \mid id \\ U &\rightarrow l \mid r \end{aligned}$$

1. 消除左递归。
2. 写 FIRST 和 FOLLOW。
3. 写预测分析表。
4. 判断消除左递归后的文法是否为 LL(1)。

2.3 三、指令调度 (15 分)

给定指令:

```

I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1

```

1. 写出指令调度的目的。
2. 写出 RAW、WAR、WAW 的定义。
3. 构建依赖图，WAR/WAW 用虚线表示。
4. 在允许寄存器重命名的条件下给出一个合法重排序列。

2.4 四、支配问题（15 分）

CFG：双分支汇合后出口

```

Entry->A
A->B
A->C
B->D
C->D
D->E
D->F
E->Exit
F->Exit

```

1. 写出 Dom、PostDom、Dom Frontier 的定义。
2. 计算每个节点的 Dom 集和 idom。
3. 写出支配边界。
4. 说明 SSA 中 phi 函数应放在什么位置。

2.5 五、符号执行与 SAT（20 分）

公式：

$$F = ((a3 \text{ and } b3) \text{ or } c3)$$

1. 写出 Path-sensitive、Flow-sensitive、Context-sensitive 的定义。
2. 写出 CNF 的定义。
3. 使用 Tseitin 算法把 F 转成 CNF。
4. 证明 2025 卷中的两组公式同时可满足。

2.6 六、Andersen 指针分析（20 分）

程序：

```
p = &A
q = &B
r = &C
*p = q
q = r
t = *p
```

1. 对每条语句写 Andersen 约束。
2. 根据约束构建闭包。
3. 写出最终 points-to 集合。

2.7 七、PPT 新增重点：SSA 与 phi（10 分）

CFG：

```
B1->B2
B1->B3
B2->B4
B3->B4

B2: x = 1
B3: x = 2
B4: y = x + 1
```

1. 说明 SSA 的基本要求。
2. 判断在哪里需要插入 x 的 phi。
3. 写出 SSA 形式的 B2、B3、B4。

3 B 详解

3.1 一、词法分析详解

正则表达式：(a|b)* a a

Thompson NFA: 起点 q0, 终点 q11。按下列转移表画图, q11 画双圈。

```
q4 --a--> q5
q6 --b--> q7
q2 --epsilon--> q4
```

```

q2 --epsilon--> q6
q5 --epsilon--> q3
q7 --epsilon--> q3
q0 --epsilon--> q2
q0 --epsilon--> q1
q3 --epsilon--> q2
q3 --epsilon--> q1
q8 --a--> q9
q1 --epsilon--> q8
q10 --a--> q11
q9 --epsilon--> q10

```

子集构造 DFA:

D0:

```

NFA集合 = {q0,q1,q2,q4,q6,q8}
on a -> D1
on b -> D2
终态 = 否

```

D1:

```

NFA集合 = {q1,q2,q3,q4,q5,q6,q8,q9,q10}
on a -> D3
on b -> D2
终态 = 否

```

D2:

```

NFA集合 = {q1,q2,q3,q4,q6,q7,q8}
on a -> D1
on b -> D2
终态 = 否

```

D3:

```

NFA集合 = {q1,q2,q3,q4,q5,q6,q8,q9,q10,q11}
on a -> D3
on b -> D2
终态 = 是

```

最小化:

初分组:

```

终态组 = {D3}
非终态组 = {D0,D1,D2}

```

最终分组:

```

G1 = {D3}
G2 = {D0,D2}
G3 = {D1}

```

每个最终分组对应一个最小 DFA 状态。

3.2 二、LL 语法分析详解

原文法:

```

R -> T U
T -> T p | id
U -> l | r

```

消除左递归:

```
R  -> T U
T  -> id T'
T' -> p T' | epsilon
U  -> l | r
```

FIRST:

```
FIRST(R)={ id }
FIRST(T)={ id }
FIRST(T')={ p, epsilon }
FIRST(U)={ l, r }
```

FOLLOW:

```
FOLLOW(R)={ $ }
FOLLOW(T)={ l, r }
FOLLOW(T')={ l, r }
FOLLOW(U)={ $ }
```

预测表:

```
M[R,id] = R -> T U
M[T,id] = T -> id T'
M[T',p] = T' -> p T'
M[T',l] = T' -> epsilon
M[T',r] = T' -> epsilon
M[U,l] = U -> l
M[U,r] = U -> r
```

结论: 表中无多重入口, 消除左递归后的文法是 LL(1)。

3.3 三、指令调度详解

定义:

指令调度在不改变程序语义的前提下重排指令, 减少流水线停顿, 提高并行执行效率。

RAW: 先写后读, 必须保持顺序。

WAR: 先读后写, 写者不能提前。

WAW: 先写后写, 顺序影响最终值。

指令:

```
I1: R1 = R2 + R3
I2: R4 = R1 + R5
I3: R2 = R6 + R7
I4: R1 = R8 + R9
I5: R10 = R4 + R1
```

依赖:

```
I1 -> I2 RAW on R1
I1 -> I3 WAR on R2
I1 -> I4 WAW on R1
I2 -> I4 WAR on R1
```

I2 -> I5 RAW on R4
I4 -> I5 RAW on R1

寄存器重命名:

I3: R2_new = R6 + R7
I4: R1_new = R8 + R9
I5 使用 R1_new

重命名后保留 RAW, WAR/WAW 被消除。一个合法调度序列:

I1, I3, I4, I2, I5

3.4 四、支配问题详解

定义:

Dom(n): 入口到 n 的每条路径都经过的节点集合。

PostDom(n): n 到出口的每条路径都经过的节点集合。

Dom Frontier(n): n 支配某个前驱, 但不严格支配该节点的位置。

CFG 边:

Entry->A
A->B
A->C
B->D
C->D
D->E
D->F
E->Exit
F->Exit

Dom 集:

Dom(Entry)={Entry}
Dom(A)={Entry, A}
Dom(B)={Entry, A, B}
Dom(C)={Entry, A, C}
Dom(D)={Entry, A, D}
Dom(E)={Entry, A, D, E}
Dom(F)={Entry, A, D, F}
Dom(Exit)={Entry, A, D, Exit}

直接支配者:

idom(A)=Entry
idom(B)=A
idom(C)=A
idom(D)=A
idom(E)=D
idom(F)=D
idom(Exit)=D

支配边界:

```

DF(B)={D}
DF(C)={D}
DF(E)={Exit}
DF(F)={Exit}
其余节点 DF 为空集

```

SSA phi 放置: 若变量在多个基本块中被定义, 则在这些定义块的迭代支配边界处插入该变量的 phi。直观上, phi 放在多

3.5 五、符号执行与 SAT 详解

公式: $F = ((a3 \text{ and } b3) \text{ or } c3)$

引入: $x31 \leftrightarrow (a3 \text{ and } b3)$, $x32 \leftrightarrow (x31 \text{ or } c3)$, 根变量 $x32$ 为真。

```

CNF:
(not x31 or a3)
(not x31 or b3)
(x31 or not a3 or not b3)
(x32 or not x31)
(x32 or not c3)
(not x32 or x31 or c3)
(x32)

```

敏感性定义:

Path-sensitive 区分不同路径。

Flow-sensitive 区分程序点和语句顺序。

Context-sensitive 区分函数调用上下文。

同时可满足证明套用:

$F1 = (a0 \text{ or } \dots \text{ or } a_n \text{ or } c) \text{ and } (b0 \text{ or } \dots \text{ or } b_m \text{ or } \text{not } c)$, $F2 = (a0 \text{ or } \dots \text{ or } a_n \text{ or } b0 \text{ or } \dots \text{ or } b_m)$ 。

$F1$ 为真时, $c = \text{false}$ 推出某个 $a_i = \text{true}$, $c = \text{true}$ 推出某个 $b_j = \text{true}$, 所以 $F2$ 为真。

$F2$ 为真时, 若某个 $a_i = \text{true}$ 令 $c = \text{false}$; 若某个 $b_j = \text{true}$ 令 $c = \text{true}$, 所以 $F1$ 为真。

3.6 六、Andersen 指针分析详解

程序:

```

p = &A
q = &B
r = &C
*p = q
q = r
t = *p

```

约束:

```

A in pts(p)
B in pts(q)
C in pts(r)
for v in pts(p), pts(q) subset pts(v)
pts(r) subset pts(q)
for v in pts(p), pts(v) subset pts(t)

```


闭包:

`pts(p)={A}`

`pts(q)={B,C}`

`pts(r)={C}`

由 `*p=q` 得 `pts(A)={B,C}`

由 `t=*p` 得 `pts(t)={B,C}`

3.7 七、PPT 新增重点详解

SSA: Static Single Assignment, 每个变量只有一个定义。

在控制流汇合处, 如果同一个变量可能来自多个定义, 需要插入 `phi`。

`x` 在 B2 和 B3 中分别定义, 二者在 B4 汇合。

因此在 B4 插入 `x` 的 `phi`。

B2:

`x1 = 1`

B3:

`x2 = 2`

B4:

`x3 = phi(x1 from B2, x2 from B3)`

`y1 = x3 + 1`

4 C 复盘清单

词法: Thompson、epsilon-closure、move、终态集合、最小化分组。

LL: 左递归、FIRST、FOLLOW、预测表、多重入口。

IR/SSA: 三地址码、基本块、支配边界、`phi` 插入与转出。

数据流: 方向、meet、transfer、gen/kill、worklist、IN/OUT。

后端: 目标代码、地址模式、冲突图、spill、指令调度。

符号执行: 敏感性、CNF、Tseitin 子句、同时可满足证明。

过程间/指针: 调用上下文、上下文敏感、Andersen 约束与闭包。